How Apache Kafka Actually Achieves Exactly-Once Semantics

Architectural decomposition based on the engineering dissection by **Arpit Bhayani** (*Asli Engineering*, Video ID: `iRhDCIzhm4A`).

> **The Fundamental Caveat: Delivery vs. Processing**
> A widespread misconception across industry systems engineering is that Kafka provides end-to-end "exactly-once processing" across external side-effects. In reality, **Kafka guarantees Exactly-Once Delivery (EOS) to its log storage**. Kafka has zero visibility into downstream consumer execution side-effects (e.g., REST calls or database writes). End-to-end exactly-once computation requires downstream idempotency or transactional sink coordination.

1. The Problem Space: The Fallacy of At-Least-Once Delivery

By default, distributed message brokers provide **at-least-once delivery**. In a network partition or transient latency spike, the classical *Two Generals' Paradox* manifests:

The ACK Loss Failure Mode — ● Duplicate Risk

1. Producer produces record batch *B* to broker.
2. Broker persists *B* to disk commit log.
3. Broker sends network `ACK` back to Producer.
4. **Failure:** Network drops `ACK` or Producer times out.
5. Producer initiates retry; broker appends *B* again, causing duplicates.

The 3-Layer Solution Hierarchy — Kafka EOS Stack

Layer 1: Producer Idempotence (Deduplication via PID & Sequence Numbers within a session).
Layer 2: Transactional IDs & Epochs (Zombie fencing across crashes and restarts).
Layer 3: 2-Phase Commit Coordinator (Atomic multi-partition writes via `__transaction_state`).

2. Delivery Semantics Comparison Matrix

Semantic Model	Producer Configuration	Broker Behavior on Retries	Failure Trade-Off
At-Most-Once	`acks=0`, retries disabled	Fire and forget; no persistence confirmation.	Zero duplicates, high potential data loss on disconnect.
At-Least-Once	`acks=all`, retries enabled, idempotence off	Broker appends every retry to commit log as a new offset.	Zero data loss, high risk of duplicate records in log.
Exactly-Once	`enable.idempotence=true`, `transactional.id` configured	Broker compares sequence buffer and filters duplicate batches.	Minimal throughput overhead (~3%), eliminates duplicate writes and zombie splits.

3. Layer 1: Producer Idempotence (Deduplicating Single-Session Retries)

Producer idempotence prevents duplicate writes arising from network transport timeouts. Kafka achieves this without inspecting the message payload (which would introduce immense CPU overhead and break intentional duplicate events). Instead, it uses protocol-level metadata:



Figure 1: Broker-side deduplication pipeline using unique Producer ID (PID) and per-partition sequence checking.

Producer ID (PID) & Sequence Number

Upon bootstrap with `enable.idempotence=true`, the broker assigns the client an internal 64-bit `PID`. For every partition target, the client maintains a monotonically increasing sequence counter (starting at `0`) appended to record batch headers.

Broker Evaluation Rules

Next Expected (Seq = Last + 1): Appends to commit log and increments cache pointer.
Duplicate (Seq ≤ Last): Ignores payload, returns existing ACK.
Out of Order (Seq > Last + 1): Rejects batch with `OutOfOrderSequenceException`.

4. Layer 2: Transactional IDs & Zombie Fencing

Idempotence alone fails when a producer process crashes, experiences a deep JVM Stop-The-World garbage collection pause, or triggers network isolation. When a replacement producer boots up, it obtains a *new random PID*. The stale "zombie" producer may wake up later and interleave duplicate messages.

> **The Solution: Stable Transactional IDs + Monotonic Epochs**
> Engineers define an explicit logical identifier (e.g., `transactional.id="order-engine-us-east"`) mapped across process lifecycles. When a producer initializes via `initTransactions()`, the Transaction Coordinator assigns it the same logical identity but increments its **Producer Epoch**. Any concurrent or delayed write stamped with an older epoch is summarily rejected with a `ProducerFencedException`, forcibly terminating the zombie process.

5. Layer 3: Atomic Multi-Partition Writes via 2-Phase Commit (2PC)

In streaming applications (such as read-process-write loops in Kafka Streams), a single transaction must atomically publish to multiple output topic-partitions while simultaneously committing the consumer input offsets to `__consumer_offsets`.



Figure 2: Kafka's modified lightweight Two-Phase Commit protocol for multi-partition atomic boundaries.

Internal Transaction State Topic

State transitions are stored in an internal append-only topic named `__transaction_state` (configured with 50 partitions). The Transaction Coordinator for a specific producer is elected by hashing its `transactional.id` to one of these 50 partitions.

Control Markers & Consumer Isolation

Commit/Abort markers are control records appended directly to partition logs. Downstream consumers configured with `isolation.level=read_committed` buffer uncommitted messages and only release them to client code once a `COMMIT` marker is encountered.

6. Architectural Summary & Key Engineering Takeaways

The Complete Kafka Exactly-Once Guarantee Paradigm

1. **Retries without duplicates:** PID + Monotonic Sequence numbers eliminate network-drop duplicate writes.
2. **Crash recovery without zombies:** `transactional.id` + Producer Epochs immediately fence old split-brain instances.
3. **Multi-partition atomicity:** 2PC via `__transaction_state` guarantees all-or-nothing writes across partitions.
4. **Application responsibility:** Kafka guarantees *exactly-once delivery to brokers*. Ensuring end-to-end exactly-once business logic requires consumers to maintain idempotent updates in downstream datastores.